In [4]:
```
import os

print("="*70)
print("Setting up Financial Sentiment Analysis Project")
print("="*70 + "\n")

# Create folder structure
os.makedirs('financial-sentiment-analysis', exist_ok=True)
os.makedirs('financial-sentiment-analysis/data', exist_ok=True)
os.makedirs('financial-sentiment-analysis/models', exist_ok=True)
os.makedirs('financial-sentiment-analysis/results', exist_ok=True)

print("✓ Folder structure created:")
print("  financial-sentiment-analysis/")
print("     ├──  data/")
print("     ├──  models/")
print("     └──  results/")
```

```
======================================================================
Setting up Financial Sentiment Analysis Project
======================================================================

✓ Folder structure created:
  financial-sentiment-analysis/
     ├──  data/
     ├──  models/
     └──  results/
```

In [5]:
```
import subprocess
import sys

print("\n[1] Installing packages...")
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "transfor
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "torch"])
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "scikit-l
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "nltk"])

print("✓ All packages installed!")
```

```
[1] Installing packages...
✓ All packages installed!
```

In [6]:
```
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModel


class FinancialEncoder:
    """
    Contextual embedding encoder for financial sentiment analysis.
    Uses DistilBERT with CLS and Mean pooling strategies.
    """

    def __init__(self, model_name="distilbert-base-uncased", pooling_strategy=
```

```
        """
        Initialize the encoder.

        Args:
            model_name: HuggingFace model identifier
            pooling_strategy: "cls" or "mean"
        """
        self.model_name = model_name
        self.pooling_strategy = pooling_strategy.lower()

        if self.pooling_strategy not in ["cls", "mean"]:
            raise ValueError("pooling_strategy must be 'cls' or 'mean'")

        print(f"Loading {model_name} with {pooling_strategy} pooling...")

        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModel.from_pretrained(model_name)
        self.model.eval()

        self.embedding_dim = self.model.config.hidden_size
        print(f"  Embedding dimension: {self.embedding_dim}")

        self.device = torch.device("cuda" if torch.cuda.is_available() else "c
        self.model.to(self.device)
        print(f"✓ Model loaded on device: {self.device}\n")

    def encode(self, texts):
        """
        Encode a list of texts to contextual embeddings.

        Args:
            texts: List of text strings

        Returns:
            numpy array of shape (N, embedding_dim)
        """
        embeddings = []

        with torch.no_grad():
            for i, text in enumerate(texts):
                if (i + 1) % max(1, len(texts) // 10) == 0:
                    print(f"  Encoding {i + 1}/{len(texts)}...")

                encoded = self.tokenizer(
                    text,
                    return_tensors="pt",
                    padding=True,
                    truncation=True,
                    max_length=512
                )

                input_ids = encoded["input_ids"].to(self.device)
                attention_mask = encoded["attention_mask"].to(self.device)
```

```
                outputs = self.model(
                    input_ids=input_ids,
                    attention_mask=attention_mask,
                    output_hidden_states=False
                )

                if self.pooling_strategy == "cls":
                    # CLS pooling: use [CLS] token (first token)
                    embedding = outputs.last_hidden_state[:, 0, :].cpu().numpy
                else:
                    # Mean pooling: average all tokens weighted by attention m
                    last_hidden_state = outputs.last_hidden_state
                    attention_mask_expanded = attention_mask.unsqueeze(-1).exp
                    sum_embeddings = (last_hidden_state * attention_mask_expar
                    sum_mask = attention_mask_expanded.sum(dim=1)
                    embedding = (sum_embeddings / sum_mask).cpu().numpy()

                embeddings.append(embedding.squeeze())

        return np.array(embeddings)

    def get_disambiguation_probe(self):
        """
        Disambiguation probe: Test contextual understanding of "bank".
        Compares embeddings in financial vs. geographic contexts.

        Returns:
            dict with cosine_similarity metric
        """
        sentences = [
            "The bank raised interest rates.",
            "She sat by the river bank."
        ]

        print(f"\nDisambiguation Probe ({self.pooling_strategy.upper()} poolir
        embeddings = self.encode(sentences)

        emb1 = embeddings[0]
        emb2 = embeddings[1]

        cosine_sim = np.dot(emb1, emb2) / (np.linalg.norm(emb1) * np.linalg.nc

        print(f"  Sentence 1: '{sentences[0]}'")
        print(f"  Sentence 2: '{sentences[1]}'")
        print(f"  Cosine Similarity: {cosine_sim:.4f}")
        print(f"  → Lower similarity = Better context understanding\n")

        return {"cosine_similarity": float(cosine_sim)}


print("✓ FinancialEncoder class defined")
```

```
✓ FinancialEncoder class defined
```

In [7]:
```
import numpy as np
import torch
import torch.nn as nn
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.metrics import f1_score, confusion_matrix, classification_report
from transformers import AutoTokenizer, AutoModel, get_linear_schedule_with_wa
from torch.optim import AdamW
from torch.utils.data import DataLoader, TensorDataset


class MLPClassifier(nn.Module):
    """Two-layer MLP classifier: 256 → 128 → 3"""

    def __init__(self, input_dim=256, hidden_dim=128, num_classes=3, dropout=0
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.fc2 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x


class SentimentPipeline:
    """Complete sentiment analysis pipeline with all four setups"""

    def __init__(self, model_name="distilbert-base-uncased"):
        self.model_name = model_name
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.encoder = AutoModel.from_pretrained(model_name)
        self.device = torch.device("cuda" if torch.cuda.is_available() else "c
        self.encoder.to(self.device)
        self.encoder.eval()

        self.embedding_dim = self.encoder.config.hidden_size
        print(f"✓ Pipeline: embedding_dim={self.embedding_dim}\n")

    def get_embeddings(self, texts, pooling_strategy="cls"):
        """Extract embeddings using specified pooling strategy"""
        embeddings = []

        with torch.no_grad():
            for i, text in enumerate(texts):
                if (i + 1) % max(1, len(texts) // 10) == 0:
                    print(f"    {i + 1}/{len(texts)}...", end='\r')

                encoded = self.tokenizer(
```

```
                    text,
                    return_tensors="pt",
                    padding=True,
                    truncation=True,
                    max_length=512
                )

                input_ids = encoded["input_ids"].to(self.device)
                attention_mask = encoded["attention_mask"].to(self.device)

                outputs = self.encoder(input_ids=input_ids, attention_mask=att

                if pooling_strategy == "cls":
                    embedding = outputs.last_hidden_state[:, 0, :].cpu().numpy
                else:
                    last_hidden_state = outputs.last_hidden_state
                    attention_mask_expanded = attention_mask.unsqueeze(-1).exp
                    sum_embeddings = (last_hidden_state * attention_mask_expar
                    sum_mask = attention_mask_expanded.sum(dim=1)
                    embedding = (sum_embeddings / sum_mask).cpu().numpy()

                embeddings.append(embedding.squeeze())

        print("                    ", end='\r')
        return np.array(embeddings)

    def setup_1_frozen_cls_logistic(self, X_train, y_train, X_test, y_test):
        """Setup 1: Frozen CLS + Logistic Regression"""
        print("\n" + "="*70)
        print("Setup 1: Frozen CLS + Logistic Regression")
        print("="*70)

        print("Extracting CLS embeddings...")
        train_embeddings = self.get_embeddings(X_train, pooling_strategy="cls"
        test_embeddings = self.get_embeddings(X_test, pooling_strategy="cls")

        print("Training Logistic Regression...")
        clf = LogisticRegression(max_iter=1000, random_state=42, n_jobs=-1)
        clf.fit(train_embeddings, y_train)

        y_pred = clf.predict(test_embeddings)
        macro_f1 = f1_score(y_test, y_pred, average="macro")
        cm = confusion_matrix(y_test, y_pred)

        print(f"\nMacro F1-Score: {macro_f1:.4f}")
        print("\nConfusion Matrix:")
        print(cm)
        print("\nClassification Report:")
        print(classification_report(y_test, y_pred, target_names=["negative",

        return {"macro_f1": macro_f1, "confusion_matrix": cm, "y_pred": y_pred

    def setup_2_frozen_mean_logistic(self, X_train, y_train, X_test, y_test):
```

```
        """Setup 2: Frozen Mean Pooling + Logistic Regression"""
        print("\n" + "="*70)
        print("Setup 2: Frozen Mean Pooling + Logistic Regression")
        print("="*70)

        print("Extracting Mean Pooling embeddings...")
        train_embeddings = self.get_embeddings(X_train, pooling_strategy="mean
        test_embeddings = self.get_embeddings(X_test, pooling_strategy="mean")

        print("Training Logistic Regression...")
        clf = LogisticRegression(max_iter=1000, random_state=42, n_jobs=-1)
        clf.fit(train_embeddings, y_train)

        y_pred = clf.predict(test_embeddings)
        macro_f1 = f1_score(y_test, y_pred, average="macro")
        cm = confusion_matrix(y_test, y_pred)

        print(f"\nMacro F1-Score: {macro_f1:.4f}")
        print("\nConfusion Matrix:")
        print(cm)
        print("\nClassification Report:")
        print(classification_report(y_test, y_pred, target_names=["negative",

        return {"macro_f1": macro_f1, "confusion_matrix": cm, "y_pred": y_pred

    def setup_3_frozen_mean_mlp(self, X_train, y_train, X_test, y_test, epochs
        """Setup 3: Frozen Mean Pooling + MLP (256 → 128 → 3)"""
        print("\n" + "="*70)
        print("Setup 3: Frozen Mean Pooling + MLP (256 → 128 → 3)")
        print("="*70)

        print("Extracting Mean Pooling embeddings...")
        train_embeddings = self.get_embeddings(X_train, pooling_strategy="mean
        test_embeddings = self.get_embeddings(X_test, pooling_strategy="mean")

        # Dimensionality reduction: 768 → 256
        print("Reducing dimensionality to 256 with PCA...")
        pca = PCA(n_components=256)
        train_embeddings = pca.fit_transform(train_embeddings)
        test_embeddings = pca.transform(test_embeddings)

        train_dataset = TensorDataset(
            torch.FloatTensor(train_embeddings),
            torch.LongTensor(y_train)
        )
        test_dataset = TensorDataset(
            torch.FloatTensor(test_embeddings),
            torch.LongTensor(y_test)
        )

        train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
        test_loader = DataLoader(test_dataset, batch_size=32)
```

```
        mlp = MLPClassifier(input_dim=256, hidden_dim=128, num_classes=3, drop
        optimizer = torch.optim.Adam(mlp.parameters(), lr=0.001)
        criterion = nn.CrossEntropyLoss()

        print(f"Training MLP for {epochs} epochs...")
        for epoch in range(epochs):
            mlp.train()
            total_loss = 0
            for embeddings, labels in train_loader:
                embeddings = embeddings.to(self.device)
                labels = labels.to(self.device)

                optimizer.zero_grad()
                outputs = mlp(embeddings)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
                total_loss += loss.item()

            print(f"  Epoch {epoch + 1}/{epochs}, Loss: {total_loss / len(trai

        mlp.eval()
        all_preds = []
        with torch.no_grad():
            for embeddings, labels in test_loader:
                embeddings = embeddings.to(self.device)
                outputs = mlp(embeddings)
                preds = torch.argmax(outputs, dim=1)
                all_preds.extend(preds.cpu().numpy())

        y_pred = np.array(all_preds)
        macro_f1 = f1_score(y_test, y_pred, average="macro")
        cm = confusion_matrix(y_test, y_pred)

        print(f"\nMacro F1-Score: {macro_f1:.4f}")
        print("\nConfusion Matrix:")
        print(cm)
        print("\nClassification Report:")
        print(classification_report(y_test, y_pred, target_names=["negative",

        return {"macro_f1": macro_f1, "confusion_matrix": cm, "y_pred": y_pred

    def setup_4_end_to_end_finetune(self, X_train, y_train, X_test, y_test, ep
        """Setup 4: End-to-End Fine-Tuning (AdamW, lr=2e-5, 5 epochs)"""
        print("\n" + "="*70)
        print("Setup 4: End-to-End Fine-Tuning (AdamW, lr=2e-5, 5 epochs)")
        print("="*70)

        print("Tokenizing data...")
        train_encoded = self.tokenizer(
            X_train,
            return_tensors="pt",
            padding=True,
```

```
            truncation=True,
            max_length=512
        )
        test_encoded = self.tokenizer(
            X_test,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=512
        )

        train_dataset = TensorDataset(
            train_encoded["input_ids"],
            train_encoded["attention_mask"],
            torch.LongTensor(y_train)
        )
        test_dataset = TensorDataset(
            test_encoded["input_ids"],
            test_encoded["attention_mask"],
            torch.LongTensor(y_test)
        )

        train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
        test_loader = DataLoader(test_dataset, batch_size=16)

        print("Loading encoder for fine-tuning...")
        encoder = AutoModel.from_pretrained(self.model_name).to(self.device)
        classifier = nn.Linear(self.embedding_dim, 3).to(self.device)

        optimizer = AdamW(
            list(encoder.parameters()) + list(classifier.parameters()),
            lr=lr
        )
        total_steps = len(train_loader) * epochs
        scheduler = get_linear_schedule_with_warmup(
            optimizer,
            num_warmup_steps=0,
            num_training_steps=total_steps
        )
        criterion = nn.CrossEntropyLoss()

        print(f"Fine-tuning for {epochs} epochs with lr={lr}...")
        for epoch in range(epochs):
            encoder.train()
            classifier.train()
            total_loss = 0

            for input_ids, attention_mask, labels in train_loader:
                input_ids = input_ids.to(self.device)
                attention_mask = attention_mask.to(self.device)
                labels = labels.to(self.device)

                outputs = encoder(input_ids=input_ids, attention_mask=attentic
```

```
                attention_mask_expanded = attention_mask.unsqueeze(-1).expand(
                sum_embeddings = (outputs.last_hidden_state * attention_mask_e
                sum_mask = attention_mask_expanded.sum(dim=1)
                embeddings = sum_embeddings / sum_mask

                logits = classifier(embeddings)
                loss = criterion(logits, labels)

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
                scheduler.step()
                total_loss += loss.item()

            print(f"  Epoch {epoch + 1}/{epochs}, Loss: {total_loss / len(trai

        encoder.eval()
        classifier.eval()
        all_preds = []

        with torch.no_grad():
            for input_ids, attention_mask, labels in test_loader:
                input_ids = input_ids.to(self.device)
                attention_mask = attention_mask.to(self.device)

                outputs = encoder(input_ids=input_ids, attention_mask=attentic
                attention_mask_expanded = attention_mask.unsqueeze(-1).expand(
                sum_embeddings = (outputs.last_hidden_state * attention_mask_e
                sum_mask = attention_mask_expanded.sum(dim=1)
                embeddings = sum_embeddings / sum_mask

                logits = classifier(embeddings)
                preds = torch.argmax(logits, dim=1)
                all_preds.extend(preds.cpu().numpy())

        y_pred = np.array(all_preds)
        macro_f1 = f1_score(y_test, y_pred, average="macro")
        cm = confusion_matrix(y_test, y_pred)

        print(f"\nMacro F1-Score: {macro_f1:.4f}")
        print("\nConfusion Matrix:")
        print(cm)
        print("\nClassification Report:")
        print(classification_report(y_test, y_pred, target_names=["negative",

        return {"macro_f1": macro_f1, "confusion_matrix": cm, "y_pred": y_pred


print("✓ SentimentPipeline and MLPClassifier classes defined")
```

```
✓ SentimentPipeline and MLPClassifier classes defined
```

In [8]:

```
import re
import os
```

```python
from sklearn.model_selection import train_test_split
from nltk.sentiment import SentimentIntensityAnalyzer
import pandas as pd
import nltk

# Download VADER lexicon
try:
    nltk.data.find('vader_lexicon')
except LookupError:
    nltk.download('vader_lexicon', quiet=True)


def load_financial_phrasebank(file_path):
    """Load FinancialPhraseBank dataset"""
    print(f"Loading dataset from: {file_path}\n")

    texts = []
    labels = []
    label_map = {"negative": 0, "neutral": 1, "positive": 2}

    with open(file_path, 'r', encoding='latin-1') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue

            match = re.search(r'\.@(negative|neutral|positive)\s*$', line)
            if match:
                sentiment = match.group(1)
                sentence = line[:match.start()].strip()

                if len(sentence) < 3:
                    continue

                label = label_map[sentiment]
                texts.append(sentence)
                labels.append(label)

    labels = np.array(labels)

    print(f"✓ Loaded {len(texts)} sentences")
    unique, counts = np.unique(labels, return_counts=True)
    label_names = {0: "Negative", 1: "Neutral", 2: "Positive"}
    for label_id, count in zip(unique, counts):
        pct = 100 * count / len(labels)
        print(f"  {label_names[label_id]}: {count} ({pct:.1f}%)")

    return texts, labels


def get_vader_baseline(texts, y_test):
    """VADER sentiment baseline"""
    sia = SentimentIntensityAnalyzer()
```

```
    predictions = []

    print("\nComputing VADER baseline...")
    for i, text in enumerate(texts):
        if (i + 1) % max(1, len(texts) // 10) == 0:
            print(f"  {i + 1}/{len(texts)}...", end='\r')

        scores = sia.polarity_scores(text)
        compound = scores["compound"]

        if compound < -0.1:
            predictions.append(0)
        elif compound > 0.1:
            predictions.append(2)
        else:
            predictions.append(1)

    print("                ", end='\r')
    vader_f1 = f1_score(y_test, np.array(predictions), average="macro")
    return vader_f1


# ============================================================================
# MAIN EXECUTION
# ============================================================================

print("\n" + "="*70)
print("NLP TASK-1: Financial Sentiment Analysis")
print("="*70 + "\n")

# Load data
print("[1] Loading Financial Sentiment Dataset...")
file_path = "/kaggle/input/datasets/shauryahandu/financialphrasebank-v1-0/Fina
texts, labels = load_financial_phrasebank(file_path)

# Split data
print(f"\n[2] Splitting data (80/20, seed=42)...")
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels,
    test_size=0.2,
    stratify=labels,
    random_state=42
)
print(f"✓ Train: {len(X_train)}, Test: {len(X_test)}\n")

# Phase II: Run all setups
print("="*70)
print("PHASE II: SENTIMENT CLASSIFICATION")
print("="*70)

pipeline = SentimentPipeline()
results = {}
```

```
results["setup_1"] = pipeline.setup_1_frozen_cls_logistic(X_train, y_train, X_
results["setup_2"] = pipeline.setup_2_frozen_mean_logistic(X_train, y_train, X
results["setup_3"] = pipeline.setup_3_frozen_mean_mlp(X_train, y_train, X_test
results["setup_4"] = pipeline.setup_4_end_to_end_finetune(X_train, y_train, X_

# Baselines
print("\n" + "="*70)
print("BASELINES")
print("="*70 + "\n")

vader_f1 = get_vader_baseline(X_test, y_test)
print(f"✓ VADER F1-Score: {vader_f1:.4f}")
print(f"✓ CBOW + Logistic Regression Reference: ~0.68-0.74\n")

# Summary
print("="*70)
print("SUMMARY: RESULTS")
print("="*70 + "\n")

summary_data = []
for setup, result in results.items():
    summary_data.append({
        "Setup": setup.replace("_", " ").title(),
        "Macro F1": f"{result['macro_f1']:.4f}"
    })

summary_data.append({"Setup": "VADER Baseline", "Macro F1": f"{vader_f1:.4f}"})
summary_data.append({"Setup": "CBOW Reference", "Macro F1": "0.68-0.74"})

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))

best_setup = max(results.items(), key=lambda x: x[1]['macro_f1'])
print(f"\n✓ Best Setup: {best_setup[0].replace('_', ' ').title()} (F1: {best_s

print("\n" + "="*70)
print("EVALUATION COMPLETE")
print("="*70)
```

```
================================================================================
NLP TASK-1: Financial Sentiment Analysis
================================================================================

[1] Loading Financial Sentiment Dataset...
Loading dataset from: /kaggle/input/datasets/shauryahandu/financialphrasebank-v
1-0/FinancialPhraseBank-v1.0 (1)/FinancialPhraseBank-v1.0/Sentences_50Agree.txt

✓ Loaded 4730 sentences
  Negative: 587 (12.4%)
  Neutral: 2816 (59.5%)
  Positive: 1327 (28.1%)

[2] Splitting data (80/20, seed=42)...
✓ Train: 3784, Test: 946

================================================================================
PHASE II: SENTIMENT CLASSIFICATION
================================================================================
config.json:   0%|          | 0.00/483 [00:00<?, ?B/s]
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a H
F_TOKEN to enable higher rate limits and faster downloads.
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
model.safetensors:   0%|          | 0.00/268M [00:00<?, ?B/s]
Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]
DistilBertModel LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |  |
------------------------+------------+--+-
vocab_layer_norm.weight | UNEXPECTED |  |
vocab_transform.bias    | UNEXPECTED |  |
vocab_layer_norm.bias   | UNEXPECTED |  |
vocab_transform.weight  | UNEXPECTED |  |
vocab_projector.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different task/architectu
re; not ok if you expect identical arch.
```

```
✓ Pipeline: embedding_dim=768


======================================================================
Setup 1: Frozen CLS + Logistic Regression
======================================================================
Extracting CLS embeddings...
Training Logistic Regression...

Macro F1-Score: 0.7435

Confusion Matrix:
[[ 81  24  12]
 [ 19 485  59]
 [ 12  80 174]]

Classification Report:
              precision    recall  f1-score   support

    negative       0.72      0.69      0.71       117
     neutral       0.82      0.86      0.84       563
    positive       0.71      0.65      0.68       266

    accuracy                           0.78       946
   macro avg       0.75      0.74      0.74       946
weighted avg       0.78      0.78      0.78       946


======================================================================
Setup 2: Frozen Mean Pooling + Logistic Regression
======================================================================
Extracting Mean Pooling embeddings...
Training Logistic Regression...

Macro F1-Score: 0.7583

Confusion Matrix:
[[ 80  24  13]
 [ 14 494  55]
 [ 14  70 182]]

Classification Report:
              precision    recall  f1-score   support

    negative       0.74      0.68      0.71       117
     neutral       0.84      0.88      0.86       563
    positive       0.73      0.68      0.71       266

    accuracy                           0.80       946
   macro avg       0.77      0.75      0.76       946
weighted avg       0.80      0.80      0.80       946


======================================================================
```

```
Setup 3: Frozen Mean Pooling + MLP (256 → 128 → 3)
======================================================================
Extracting Mean Pooling embeddings...
Reducing dimensionality to 256 with PCA...
Training MLP for 5 epochs...
  Epoch 1/5, Loss: 0.8382
  Epoch 2/5, Loss: 0.5314
  Epoch 3/5, Loss: 0.4338
  Epoch 4/5, Loss: 0.3864
  Epoch 5/5, Loss: 0.3474

Macro F1-Score: 0.7697

Confusion Matrix:
[[ 82  24  11]
 [ 14 494  55]
 [ 15  62 189]]

Classification Report:
              precision    recall  f1-score   support

    negative       0.74      0.70      0.72       117
     neutral       0.85      0.88      0.86       563
    positive       0.74      0.71      0.73       266

    accuracy                           0.81       946
   macro avg       0.78      0.76      0.77       946
weighted avg       0.81      0.81      0.81       946


======================================================================
Setup 4: End-to-End Fine-Tuning (AdamW, lr=2e-5, 5 epochs)
======================================================================
Tokenizing data...
Loading encoder for fine-tuning...
Loading weights:   0%|          | 0/100 [00:00<?, ?it/s]
DistilBertModel LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |  |
------------------------+------------+--+-
vocab_layer_norm.weight | UNEXPECTED |  |
vocab_transform.bias    | UNEXPECTED |  |
vocab_layer_norm.bias   | UNEXPECTED |  |
vocab_transform.weight  | UNEXPECTED |  |
vocab_projector.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different task/architectu
re; not ok if you expect identical arch.
```

```
Fine-tuning for 3 epochs with lr=2e-05...
  Epoch 1/3, Loss: 0.6054
  Epoch 2/3, Loss: 0.2833
  Epoch 3/3, Loss: 0.1754

Macro F1-Score: 0.8336

Confusion Matrix:
[[102  12   3]
 [ 18 494  51]
 [  9  49 208]]

Classification Report:
              precision    recall  f1-score   support

    negative       0.79      0.87      0.83       117
     neutral       0.89      0.88      0.88       563
    positive       0.79      0.78      0.79       266

    accuracy                           0.85       946
   macro avg       0.82      0.84      0.83       946
weighted avg       0.85      0.85      0.85       946


======================================================================
BASELINES
======================================================================


Computing VADER baseline...
✓ VADER F1-Score: 0.4796
✓ CBOW + Logistic Regression Reference: ~0.68-0.74

======================================================================
SUMMARY: RESULTS
======================================================================

         Setup  Macro F1
       Setup 1    0.7435
       Setup 2    0.7583
       Setup 3    0.7697
       Setup 4    0.8336
VADER Baseline    0.4796
CBOW Reference 0.68-0.74

✓ Best Setup: Setup 4 (F1: 0.8336)

======================================================================
EVALUATION COMPLETE
======================================================================
```